

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ
Факультет компьютерных систем и сетей
Кафедра электронных вычислительных машин
Дисциплина: Базы данных

Тема «Парк Юрского периода»
Лабораторная работа №6
Создание приложения для базы данных

Студент:
Преподаватель:

А.А. Горох
Д.В. Куприянова

МИНСК 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	4
1.1 Системные требования	4
1.2 Обзор интерфейса приложения.....	4
1.3 Отображение таблицы	4
1.4 Редактирование и удаление записей таблицы.....	6
1.5 Добавление записи в таблицу	6
1.6 SQL скрипты	7
1.7 Управление таблицами	9
1.8 Добавления таблицы	10
1.9 Удаление таблицы из базы данных	11
1.10 Экспорт в Excel.....	11
1.11 Создание резервной копии	12
ЗАКЛЮЧЕНИЕ	14
ПРИЛОЖЕНИЕ А	15

ВВЕДЕНИЕ

Современные информационные системы немыслимы без использования баз данных (БД), которые обеспечивают структурированное хранение, эффективное управление и быстрый доступ к большим объемам данных. Приложения для работы с базами данных играют ключевую роль в автоматизации бизнес-процессов, аналитике и поддержке принятия решений. Одной из наиболее популярных систем управления базами данных (СУБД) является PostgreSQL, благодаря своей надежности, открытому исходному коду и поддержке расширенных функций, таких как транзакции, триггеры и полнотекстовый поиск.

Разработанное программное обеспечение предоставляет пользователям интуитивно понятную и наглядную среду для выполнения основных и продвинутых операций с базой данных. Интерфейс приложения ориентирован как на опытных пользователей, так и на тех, кто только начинает работу с СУБД, устраняя необходимость ручного написания SQL-запросов для типовых операций.

В рамках задачи предполагается реализовать базовые операции управления данными: создание и удаление таблиц, редактирование их структуры, резервное копирование и восстановление информации. Особое внимание уделяется механизмам экспорта данных в форматы, удобные для анализа (например, Excel), а также сохранению и повторному использованию пользовательских запросов. Это позволяет повысить гибкость работы с информацией и адаптировать приложение под конкретные сценарии использования.

Разрабатываемое приложение демонстрирует интеграцию PostgreSQL с выбранным языком программирования, подчеркивая возможности взаимодействия между клиентской частью и СУБД. Важным аспектом является обеспечение отказоустойчивости через резервное копирование и реализацию транзакций для сохранения целостности данных.

1 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

1.1 Системные требования

Операционная система Linux или Windows 10 и выше, PostgreSQL, Go 1.24.0 с фреймворком Fyne.io, подключение к интернету для скачивания компилятором нужных зависимостей.

1.2 Обзор интерфейса приложения

В правой части окна представлена большая область для вывода всех сущностей выбранной таблицы. В левой части представлены список доступных таблиц и кнопки для управления выбранной таблицей или всей базой данных. На рисунке 1.1 представлен интерфейс приложения.

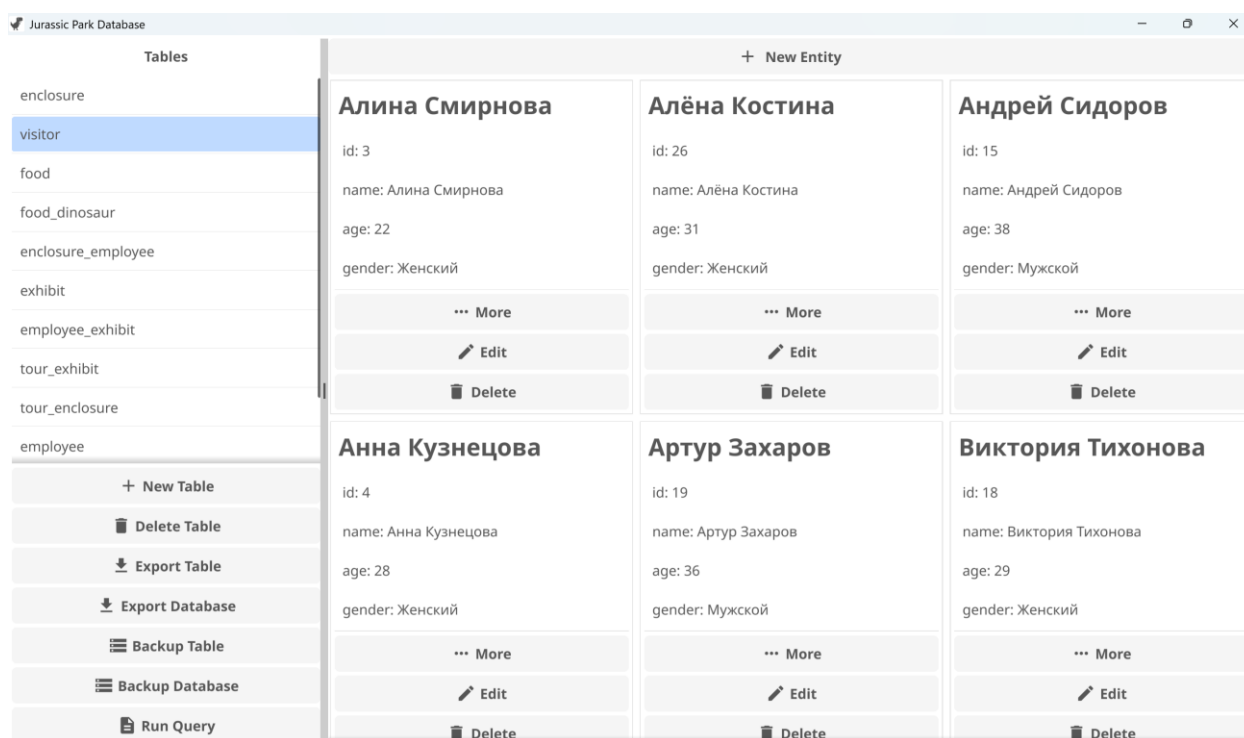


Рисунок 1.1 – Интерфейс приложения

1.3 Отображение таблицы

Для выбора интересующей таблицы для отображения необходимо выбрать таблицу из списка слева. Пример списка показан на рисунке 1.2.

После выбора таблицы, она отобразится в правой части экрана. Нажимая на кнопки «More», «Edit» и «Delete» каждой сущности таблицы, можно получить больше информации о выбранной сущности, изменить ее или удалить. Пример вывода таблицы employee представлен на рисунке 1.3.

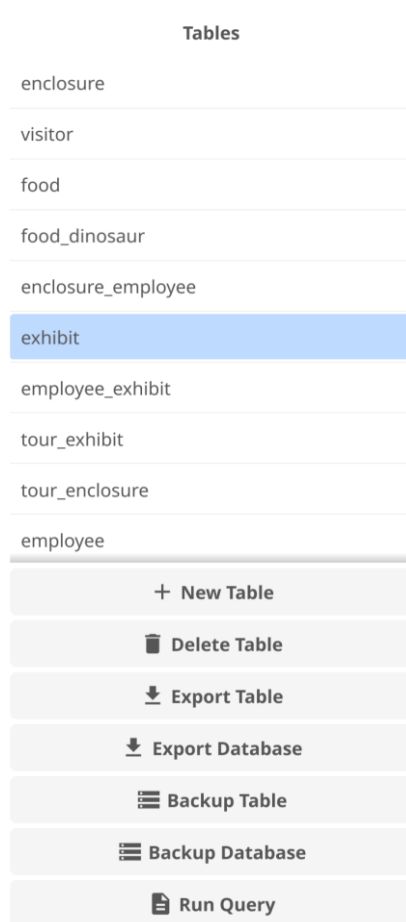


Рисунок 1.2 – Выбор таблицы

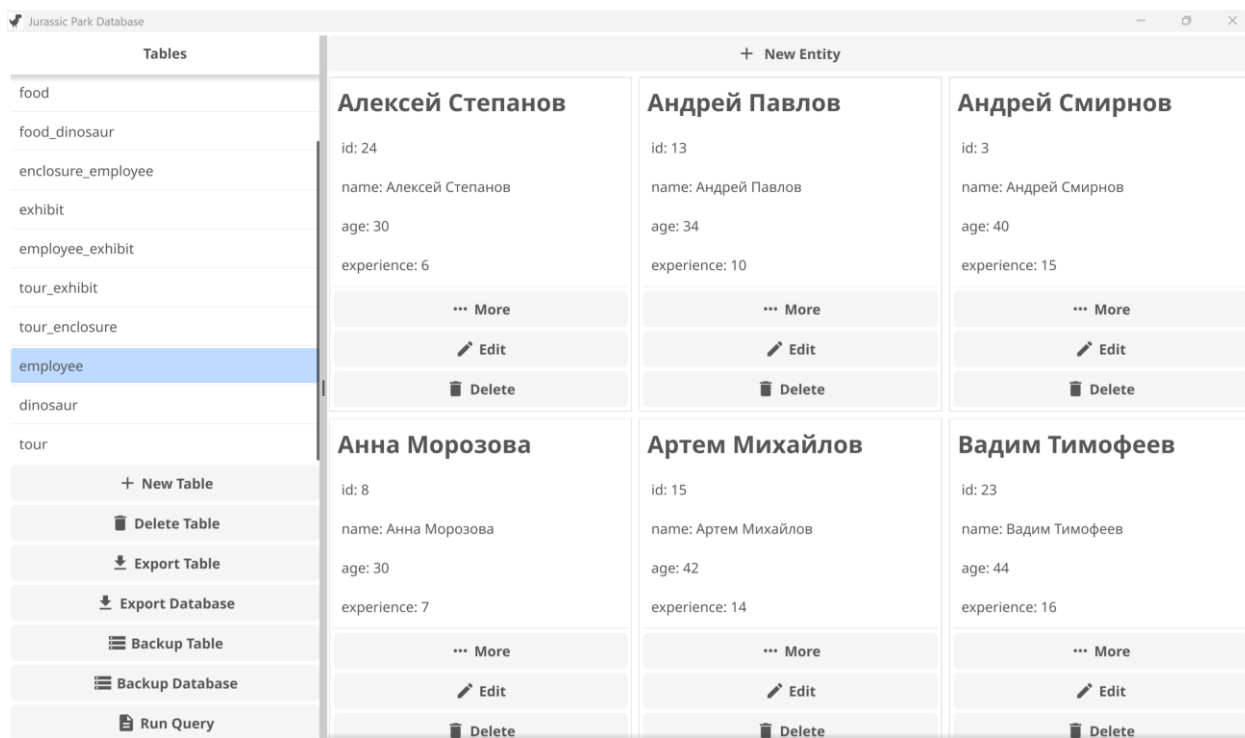


Рисунок 1.3 – Вывод таблицы employee

1.4 Редактирование и удаление записей таблицы

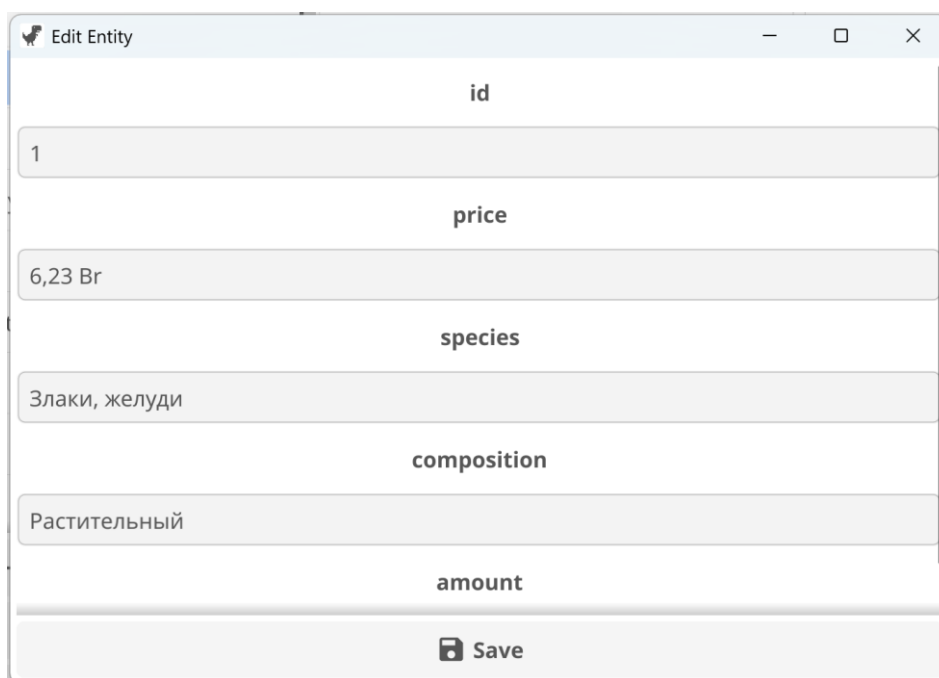
Для редактирования записи в таблице надо нажать на кнопку «Edit» на карточке нужной сущности.

После нажатия на кнопку открывается новое окно с формой для заполнения столбцов новой сущности. Для каждого столбца есть отдельное поле для ввода с названием этого столбца.

Чтобы применить изменения нужно нажать на кнопку «Save». Далее приложение сделает запрос в базу данных и обновит список сущностей.

Для удаления записи в таблице необходимо нажать на кнопку «Delete». Далее появится окно с подтверждением удаления и сущность удалится.

На рисунке 1.4 показано окно редактирования на примере записи из таблицы employee.



The screenshot shows a window titled "Edit Entity" with a standard Windows-style title bar (minimize, maximize, close buttons). The window contains a form with several input fields, each with a label above it: "id" with the value "1", "price" with the value "6,23 Br", "species" with the value "Злаки, желуди", "composition" with the value "Растительный", and "amount" which is currently empty. At the bottom of the form is a button labeled "Save" with a floppy disk icon.

Рисунок 1.4 – Окно редактирования и удаления записи в таблице

1.5 Добавление записи в таблицу

Для добавления записи в таблицу выберите нужную вам таблицу и нажмите кнопку «New entity».

После нажатия кнопки откроется окно добавления записи в таблицу. В данном окне отображается информация о каждом столбце таблицы и отдельное поле для ввода с названием этого столбца.

Внизу находится кнопка «Save» для добавления записи в таблицу. При нажатии кнопки сущность сохраняется.

На рисунке 1.5 представлено окно добавления записи в таблицу.

The image shows a window titled "New Entity" with a standard Windows-style title bar (minimize, maximize, close buttons). Inside the window, there are five text input fields stacked vertically. The first field is labeled "id", the second "price", the third "species", the fourth "composition", and the fifth "amount". Each field has a light gray border and a small vertical scrollbar on the right side. At the bottom of the window, there is a button with a floppy disk icon and the text "Save".

Рисунок 1.5 – Окно добавления записи в таблицу

1.6 SQL скрипты

В левой части окна приложения также представлена кнопка для создания и применения произвольных SQL скриптов. При нажатии на «Run query» появляется окно для выбора скрипта. На рисунке 1.9 представлено данное окно.

The image shows a window titled "Run Query" with a standard Windows-style title bar. Inside the window, there is a section labeled "Saved Queries:" followed by a dropdown menu that currently displays "(Select one)". Below this, there is a label "Query:" and a large text area with the placeholder text "Enter SQL query here...". At the bottom of the window, there are three buttons: "Save Query", "Delete Query", and "Run Query".

Рисунок 1.9 – Окно для работы с SQL скриптами

Сверху окна находится выпадающий список, в котором пользователь может выбрать уже готовый скрипт из лабораторных работ №4 и №5, а также

сохраненные пользователем скрипты. На рисунке 1.10 показан список реализованных SQL запросов из лабораторных работ №4 и №5.

После выбора скрипта под выпадающим списком в поле для ввода появится код скрипта. На рисунке 1.11 показана область после выбора скрипта. Для выполнения скрипта следует нажать кнопку «Run Query». Если скрипт был типа SELECT, то в окне приложения отобразится результат запроса.

Также можно сохранять пользовательские скрипты. Для этого надо ввести скрипт в поле ввода и нажать кнопку «Save Query». Новый скрипт сохранится в папке проекта в пакете config в файл queries.json. После этого появится окно, где надо ввести название скрипта. Окно показано на рисунке 1.12. Если скрипт с таким названием уже существует, то пользователю предложат либо поменять код скрипта, либо отменить операцию.

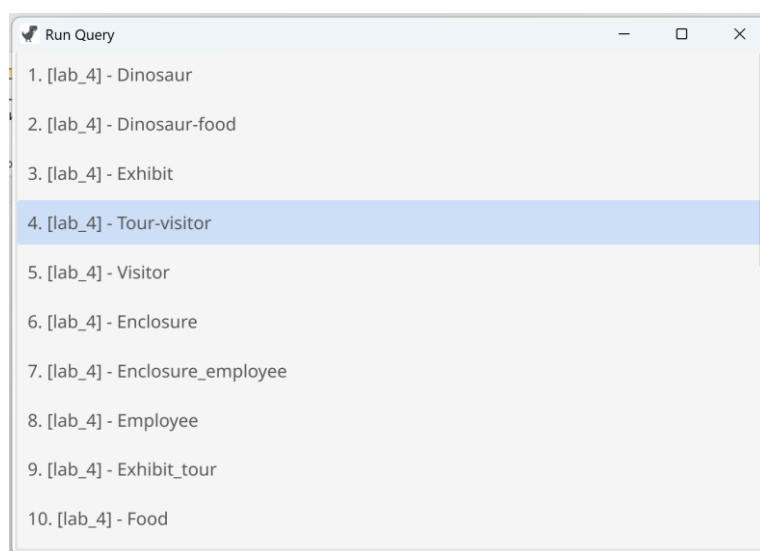


Рисунок 1.10 – Список скриптом из лабораторных работ №4 и №5

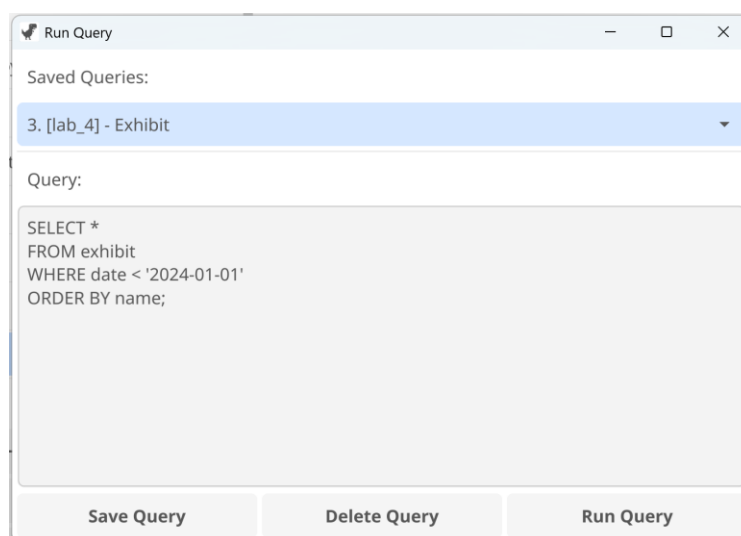


Рисунок 1.11 – Область SQL скриптов после выбора скрипта из списка

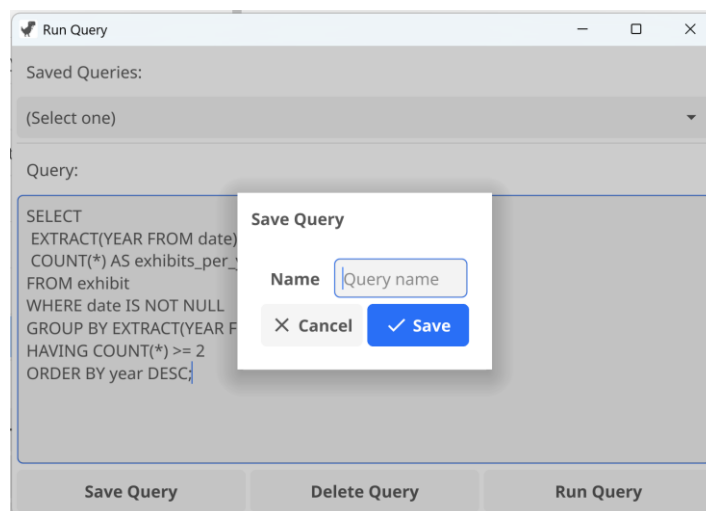


Рисунок 1.12 – Сохранение нового скрипта

Для удаления скрипта надо выбрать нужный скрипт из списка и нажать кнопку «Delete script». После нажатия кнопки появится окно с подтверждением удаления скрипта. Данное окно показано на рисунке 1.13.

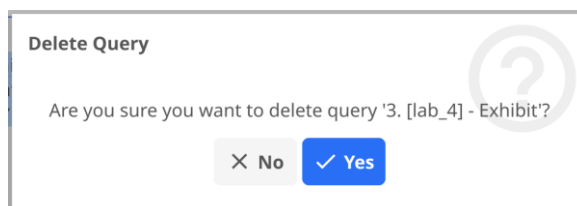


Рисунок 1.13 – Удаление скрипта

1.7 Управление таблицами

Через кнопки в левой части экрана пользователь может экспортировать текущую таблицу в Excel, создать резервную копию базы данных. На рисунке 1.14 представлена группировка кнопок для управления таблицами.

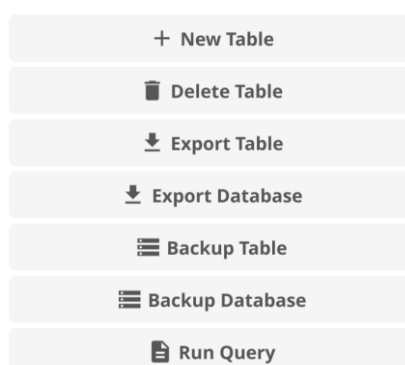


Рисунок 1.14 – Меню управления таблицами

1.8 Добавления таблицы

Для добавления таблицы в базу данных нужно нажать кнопку «New table».

После нажатия кнопки «New table» появляется окно добавления таблицы в базу данных. Содержание данного окна следующее:

- поле для ввода имени таблицы;
- поля для задания названия и типа столбца;
- кнопка добавления столбца;
- кнопка сохранения таблицы.

Типы данных столбцов:

- integer – целые числа;
- real – вещественные числа;
- text – текстовая информация;
- boolean – логический тип данных;
- date – тип данных даты;
- timestamp – тип времени (временной метки);
- money – денежный тип данных.

При нажатии кнопки «Save», если какие-то данные столбцов либо таблица с таким названием уже существует, то появиться окно с предупреждением.

На рисунке 1.6 представлено окно для добавления таблицы, на рисунке 1.7 показан выбор типа данных столбца.

The screenshot shows a 'Create Table' dialog box. It includes a 'Table Name' field, a list of columns with 'Column Name' and 'Data Type' headers, and buttons for '+ Add Column' and 'Create Table'.

Рисунок 1.6 – Окно добавления таблицы в базу данных

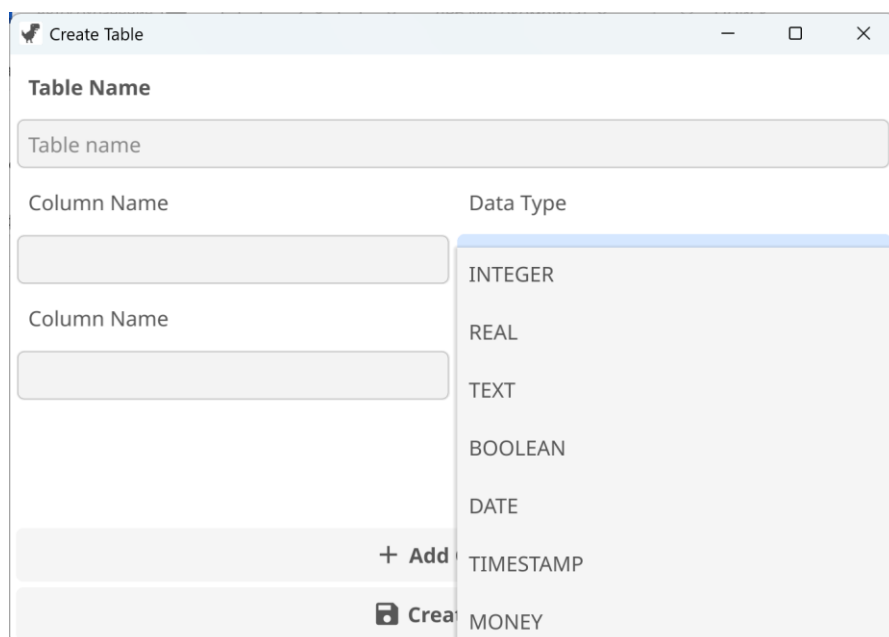


Рисунок 1.7 – Выбор типа данных для столбца

1.9 Удаление таблицы из базы данных

Для удаления таблицы выбирается нужная таблицы и нажмите кнопку «Delete table».

После нажатия кнопки «Delete table» появится окно с подтверждением удаления выбранной пользователем таблицы. Пример окна показан на рисунке 1.8.

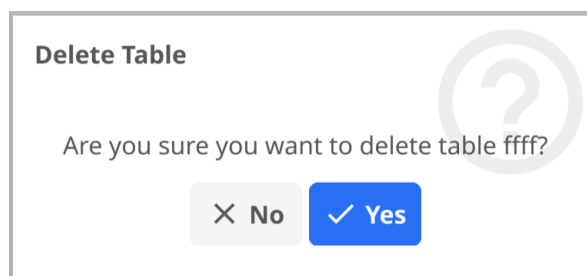


Рисунок 1.8 – Подтверждение удаления таблицы employee

1.10 Экспорт в Excel

После нажатия на кнопку «Export table» появится окно встроенного файлового редактора для сохранения Excel файла таблицы. На рисунке 1.15 продемонстрировано окно файлового менеджера. Чтобы сохранить файл, нужно выбрать местоположение и название файла и нажать «Save». Это показано на рисунке 1.16.

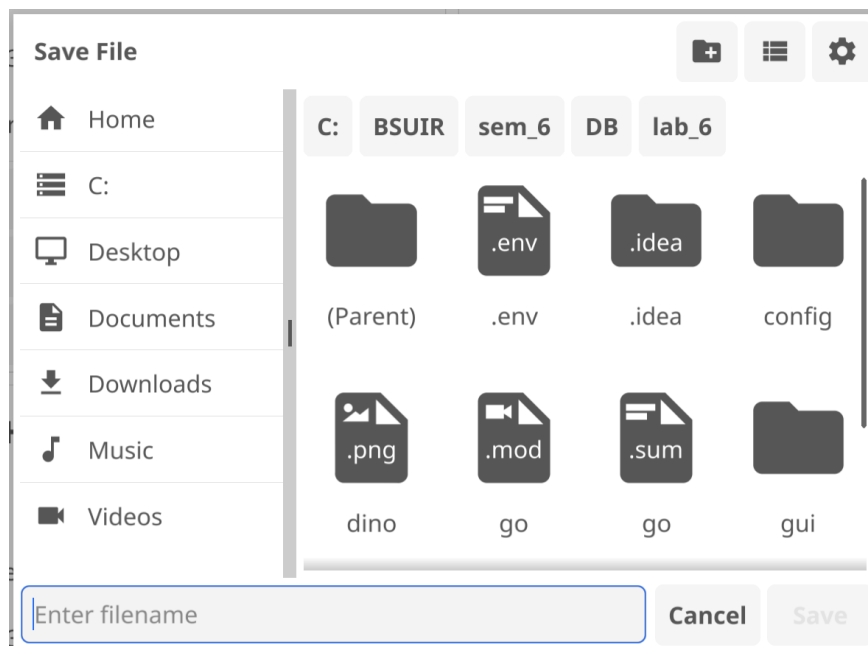


Рисунок 1.15 – Окно для экспорта в Excel

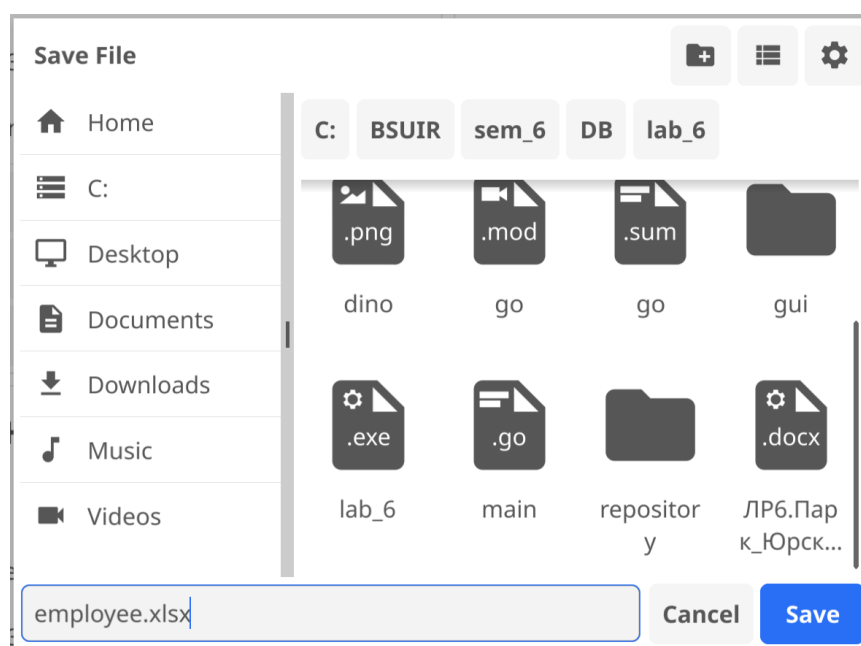


Рисунок 1.16 – Выбор места для экспорта таблицы

Соответственно экспорт всей базы данных в таблицу Excel происходит аналогично, без выбора экспортируемой таблицы.

1.11 Создание резервной копии

Для создания резервной копии таблицы или базы данных необходимо выбрать таблицу (только для «бэкапа» таблицы) и нажать на кнопку «Backup table» или «Backup database».

Для создания резервной копии нужно выбрать в всплывающем файловом менеджере местоположение и название файла резервного копирования и нажать кнопку «Save». На рисунке 1.17 показан выбор места, где сохранить файл резервной копии.

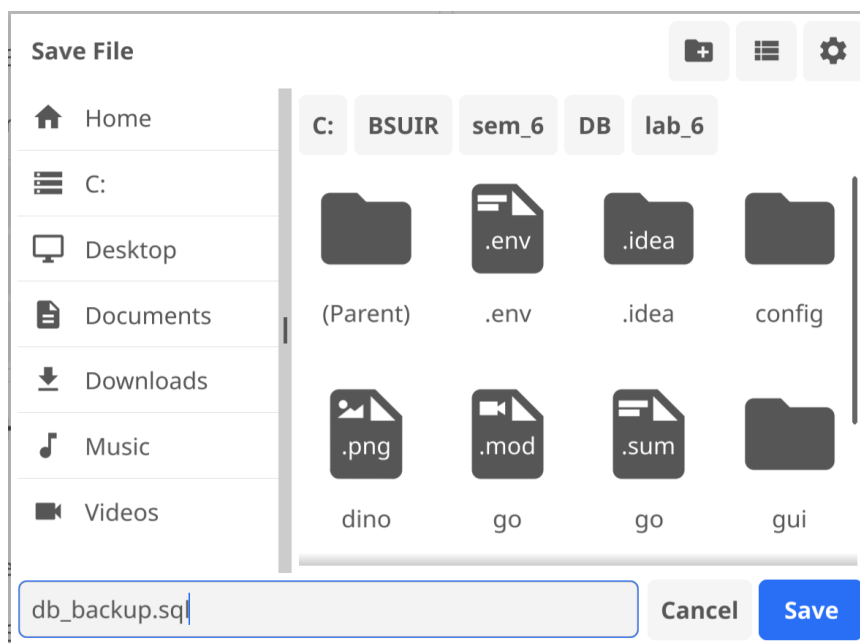


Рисунок 1.17 – Сохранение резервной копии

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы было разработано настольное приложение для взаимодействия с базой данных, написанное на языке программирования Go с использованием библиотеки Fyne.io для создания графического интерфейса. Основное назначение программы — обеспечение удобного и наглядного интерфейса для выполнения операций с базой данных PostgreSQL и визуализации результатов запросов.

Приложение реализует возможность создания, редактирования и удаления таблиц, а также управления отдельными записями. Пользователю предоставлена возможность добавления и удаления столбцов, а также работы со структурой таблиц. Кроме того, поддерживается создание резервных копий как отдельных таблиц, так и всей базы данных с возможностью последующего восстановления. Это позволяет обеспечить стабильную и безопасную работу с данными, снижая риск их потери.

Встроенный модуль управления SQL-запросами позволяет не только выполнять предопределённые запросы, но и создавать пользовательские, с возможностью их сохранения для повторного использования. Результаты запросов могут быть экспортированы в файл Excel-формата, что удобно для анализа, хранения или последующей обработки данных.

Таким образом, разработанное приложение охватывает все ключевые функции, необходимые для эффективной работы с реляционной базой данных в рамках учебного проекта. Оно сочетает в себе мощный функционал с простотой и доступностью пользовательского интерфейса, что делает его удобным как для студентов, так и для преподавателей. Структура программы также предполагает возможность дальнейшего масштабирования и функционального расширения.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Файл main.go:

```
package main

import (
    "fyne.io/fyne/v2/app"
    "github.com/joho/godotenv"
    _ "github.com/lib/pq"
    "github.com/spf13/viper"
    "lab_6/config"
    "lab_6/gui"
    "lab_6/repository"
    "log"
    "os"
)

func main() {
    if err := config.InitConfig(); err != nil {
        log.Fatal(err)
    }

    if err := godotenv.Load(".env"); err != nil {
        log.Fatalf("error loading .env variables: %s", err.Error())
    }

    pg := new(repository.PostgresRepository)
    _, err := pg.InitDB(repository.Config{
        Host:      viper.GetString("db.host"),
        Port:      viper.GetString("db.port"),
        Username:  viper.GetString("db.username"),
        DBName:    viper.GetString("db.dbname"),
        SSLMode:   viper.GetString("db.sslmode"),
        Password:  os.Getenv("DB_PASSWORD"),
    })
    if err != nil {
        log.Fatal(err)
    } else {
        log.Println("connected to the database successfully")
    }

    a := app.New()
    w := a.NewWindow("Jurassic Park Database")
    a, w = gui.GUI(pg)
    w.ShowAndRun()
}
```

Файл config.go:

```
package config

import (
    "github.com/spf13/viper"
)

func InitConfig() error {
```

```

viper.AddConfigPath("config")
viper.SetConfigName("config")
return viper.ReadInConfig()
}

```

Файл config.yml:

```

db:
  username: "postgres"
  host: "localhost"
  port: "5432"
  dbname: "jurassic-park"
  sslmode: "disable"

```

Файл postgres.go:

```

package repository

import (
    "fmt"
    "github.com/jmoiron/sqlx"
    "log"
    "strings"
)

type PostgresRepository struct {
    Db *sqlx.DB
}

type Config struct {
    Host      string
    Port      string
    Username  string
    Password  string
    DBName    string
    SSLMode   string
}

func (p *PostgresRepository) EscapeString(str string) string {
    return strings.ReplaceAll(str, "'", "''")
}

func (p *PostgresRepository) InitDB(cfg Config) (*sqlx.DB, error) {
    pg, err := sqlx.Open("postgres", fmt.Sprintf("host=%s port=%s user=%s password=%s dbname=%s sslmode=%s",
        cfg.Host,    cfg.Port,    cfg.Username,    cfg.Password,    cfg.DBName,
        cfg.SSLMode))
    if err != nil {
        log.Println(err)
        return nil, err
    }
    err = pg.Ping()
    if err != nil {
        log.Println(err)
        return nil, err
    }
    p.Db = pg
    return pg, nil
}

```

Файл repository.go:


```

package repository

import (
    "encoding/json"
    "fmt"
    "fyne.io/fyne/v2"
    _ "github.com/jmoiron/sqlx"
    "log"
    "os"
    "strings"
    "time"

    "github.com/xuri/excelize/v2"
)

const queriesFile = "config/queries.json"

type Query struct {
    Name string `json:"name"`
    SQL  string `json:"sql"`
}

type ColumnInfo struct {
    Name string
    Type string
}

func loadQueries() ([]Query, error) {
    data, err := os.ReadFile(queriesFile)
    if err != nil {
        if os.IsNotExist(err) {
            log.Println(err)
            return []Query{}, nil
        }
        log.Println(err)
        return nil, fmt.Errorf("failed to read queries file: %w", err)
    }

    var queries []Query
    if err := json.Unmarshal(data, &queries); err != nil {
        log.Println(err)
        return nil, fmt.Errorf("failed to parse queries: %w", err)
    }
    return queries, nil
}

func DeleteQuery(name string) error {
    queries, err := loadQueries()
    if err != nil {
        log.Println(err)
        return fmt.Errorf("failed to load queries: %w", err)
    }

    found := false
    for i, q := range queries {
        if q.Name == name {
            queries = append(queries[:i], queries[i+1:]...)
            found = true
            break
        }
    }

    if !found {

```

```

        return fmt.Errorf("query '%s' not found", name)
    }

    if err := saveQueries(queries); err != nil {
        log.Println(err)
        return fmt.Errorf("failed to save queries: %w", err)
    }

    log.Printf("query '%s' deleted\n", name)
    return nil
}

func saveQueries(queries []Query) error {
    data, err := json.MarshalIndent(queries, "", " ")
    if err != nil {
        log.Println(err)
        return fmt.Errorf("failed to marshal queries: %w", err)
    }
    if err := os.WriteFile(queriesFile, data, 0644); err != nil {
        log.Println(err)
        return fmt.Errorf("failed to write queries file: %w", err)
    }
    return nil
}

func GetQueryNames() ([]string, error) {
    queries, err := loadQueries()
    if err != nil {
        log.Println(err)
        return nil, err
    }
    names := make([]string, len(queries))
    for i, q := range queries {
        names[i] = q.Name
    }
    return names, nil
}

func GetQueryByName(name string) (string, error) {
    queries, err := loadQueries()
    if err != nil {
        log.Println(err)
        return "", err
    }
    for _, q := range queries {
        if q.Name == name {
            return q.SQL, nil
        }
    }
    log.Printf("query '%s' not found\n", name)
    return "", fmt.Errorf("query '%s' not found", name)
}

func SaveQuery(name, sql string) error {
    queries, err := loadQueries()
    if err != nil {
        log.Println(err)
        return err
    }

    for i, q := range queries {
        if q.Name == name {
            queries[i].SQL = sql

```

```

        return saveQueries(queries)
    }
}

queries = append(queries, Query{Name: name, SQL: sql})
log.Printf("saved query '%s'\n", name)
return saveQueries(queries)
}

func GetTableList(pg *PostgresRepository) ([]string, error) {
rows, err := pg.Db.Query(`
    SELECT table_name
    FROM information_schema.tables
    WHERE table_schema = 'public'
`)
if err != nil {
    log.Println(err)
    return nil, fmt.Errorf("failed to get tables: %w", err)
}
defer rows.Close()

var tables []string
for rows.Next() {
    var table string
    if err := rows.Scan(&table); err != nil {
        log.Println(err)
        return nil, fmt.Errorf("failed to scan table name: %w", err)
    }
    tables = append(tables, table)
}
return tables, nil
}

func GetTableEntities(pg *PostgresRepository, tableName string)
([]map[string]interface{}, error) {
    if strings.TrimSpace(tableName) == "" {
        log.Println("table name cannot be empty")
        return nil, fmt.Errorf("table name cannot be empty")
    }

    query := fmt.Sprintf("SELECT * FROM %s", tableName)
    rows, err := pg.Db.Queryx(query)
    if err != nil {
        log.Println(err)
        return nil, fmt.Errorf("failed to query table: %w", err)
    }
    defer rows.Close()

    var entities []map[string]interface{}
    for rows.Next() {
        row := make(map[string]interface{})
        if err := rows.MapScan(row); err != nil {
            log.Println(err)
            return nil, fmt.Errorf("failed to scan row: %w", err)
        }
        entities = append(entities, formatRowValues(row))
    }
    return entities, nil
}

func formatRowValues(row map[string]interface{}) map[string]interface{}
{
    for k, v := range row {

```

```

        switch val := v.(type) {
        case []byte:
            row[k] = string(val)
        case time.Time:
            if strings.Contains(strings.ToLower(k), "date") {
                row[k] = val.Format("2006-01-02")
            } else if strings.Contains(strings.ToLower(k), "time") {
                row[k] = val.Format("15:04:05")
            }
        }
    }
    return row
}

func GetTableColumns(pg *PostgresRepository, tableName string)
([]ColumnInfo, error) {
    query := `
        SELECT column_name, data_type
        FROM information_schema.columns
        WHERE table_name = $1
        ORDER BY ordinal_position
    `

    rows, err := pg.Db.Query(query, tableName)
    if err != nil {
        log.Println(err)
        return nil, fmt.Errorf("failed to get columns: %w", err)
    }
    defer rows.Close()

    var columns []ColumnInfo
    for rows.Next() {
        var col ColumnInfo
        if err := rows.Scan(&col.Name, &col.Type); err != nil {
            log.Println(err)
            return nil, fmt.Errorf("failed to scan column: %w", err)
        }
        columns = append(columns, col)
    }
    return columns, nil
}

func ExecuteQuery(pg *PostgresRepository, query string, args
...interface{}) ([]map[string]interface{}, error) {
    rows, err := pg.Db.Queryx(query, args...)
    if err != nil {
        log.Println(err)
        return nil, fmt.Errorf("query execution failed: %w", err)
    }
    defer rows.Close()

    var results []map[string]interface{}
    for rows.Next() {
        row := make(map[string]interface{})
        if err := rows.MapScan(row); err != nil {
            log.Println(err)
            return nil, fmt.Errorf("failed to scan result row: %w", err)
        }
        results = append(results, formatRowValues(row))
    }
    return results, nil
}

```

```

func DeleteTable(pg *PostgresRepository, tableName string) error {
    query := fmt.Sprintf("DROP TABLE IF EXISTS %s CASCADE", tableName)
    _, err := pg.Db.Exec(query)
    if err != nil {
        log.Println(err)
    }
    return err
}

func DeleteEntity(pg *PostgresRepository, tableName string, entity
map[string]interface{}) error {
    var whereClauses []string
    var args []interface{}
    i := 1
    for k, v := range entity {
        whereClauses = append(whereClauses, fmt.Sprintf("%s = %d", k, i))
        args = append(args, v)
        i++
    }
    query := fmt.Sprintf("DELETE FROM %s WHERE %s", tableName,
strings.Join(whereClauses, " AND "))
    _, err := pg.Db.Exec(query, args...)
    if err != nil {
        log.Println(err)
    }
    return err
}

func UpdateEntity(pg *PostgresRepository, tableName string, old
map[string]interface{}, updated map[string]interface{}) error {
    var setClauses []string
    var args []interface{}
    i := 1
    for k, v := range updated {
        setClauses = append(setClauses, fmt.Sprintf("%s = %d", k, i))
        args = append(args, v)
        i++
    }

    var whereClauses []string
    for k, v := range old {
        whereClauses = append(whereClauses, fmt.Sprintf("%s = %d", k, i))
        args = append(args, v)
        i++
    }

    query := fmt.Sprintf("UPDATE %s SET %s WHERE %s", tableName,
strings.Join(setClauses, ", "), strings.Join(whereClauses, " AND "))
    _, err := pg.Db.Exec(query, args...)
    if err != nil {
        log.Println(err)
    }
    return err
}

func InsertEntity(pg *PostgresRepository, tableName string, values
map[string]interface{}) error {
    var keys []string
    var params []string
    var args []interface{}
    i := 1
    for k, v := range values {
        keys = append(keys, k)

```

```

        params = append(params, fmt.Sprintf("%d", i))
        args = append(args, v)
        i++
    }
    query := fmt.Sprintf("INSERT INTO %s (%s) VALUES (%s)", tableName,
strings.Join(keys, ", "), strings.Join(params, ", "))
    _, err := pg.Db.Exec(query, args...)
    if err != nil {
        log.Println(err)
    }
    return err
}

func BackupTable(pg *PostgresRepository, tableName string) (string,
error) {
    var backup strings.Builder

    var tableDDL string
    err := pg.Db.Get(&tableDDL, fmt.Sprintf(`
SELECT 'CREATE TABLE ' || table_name || ' (' ||
string_agg(column_name || ' ' || data_type ||
CASE WHEN is_nullable = 'NO' THEN ' NOT NULL' ELSE '' END, ', ')
||
        ');'
FROM information_schema.columns
WHERE table_name = '%s'
GROUP BY table_name`, tableName))
    if err != nil {
        log.Println(err)
        return "", fmt.Errorf("failed to get table DDL: %w", err)
    }
    backup.WriteString(tableDDL + "\n\n")

    rows, err := pg.Db.Queryx(fmt.Sprintf("SELECT * FROM %s", tableName))
    if err != nil {
        log.Println(err)
        return "", fmt.Errorf("failed to query table data: %w", err)
    }
    defer rows.Close()

    columns, err := rows.Columns()
    if err != nil {
        log.Println(err)
        return "", fmt.Errorf("failed to get columns: %w", err)
    }

    for rows.Next() {
        row := make(map[string]interface{})
        err := rows.MapScan(row)
        if err != nil {
            log.Println(err)
            return "", fmt.Errorf("failed to scan row: %w", err)
        }

        var values []string
        for _, col := range columns {
            val := row[col]
            if val == nil {
                values = append(values, "NULL")
            } else {
                switch v := val.(type) {
                case []byte:

```

```

                                values = append(values, fmt.Sprintf("%s",
pg.EscapeString(string(v))))
                                case string:
                                    values = append(values, fmt.Sprintf("%s",
pg.EscapeString(v)))
                                case time.Time:
                                    if strings.Contains(strings.ToLower(col),
"date") {
                                        values = append(values,
fmt.Sprintf("%s", v.Format("2006-01-02")))
                                    } else if strings.Contains(strings.ToLower(col),
"time") {
                                        values = append(values,
fmt.Sprintf("%s", v.Format("15:04:05")))
                                    } else {
                                        values = append(values,
fmt.Sprintf("%s", v.Format(time.RFC3339)))
                                    }
                                default:
                                    values = append(values, fmt.Sprintf("%v", v))
                                }
                            }
                        }

                        backup.WriteString(fmt.Sprintf("INSERT INTO %s (%s) VALUES
(%s);\n",
                            tableName, strings.Join(columns, ", "), strings.Join(values,
", ")))
                    }

                    var indexes []string
                    err = pg.Db.Select(&indexes, `
                        SELECT indexdef
                        FROM pg_indexes
                        WHERE tablename = $1`, tableName)
                    if err != nil {
                        log.Println(err)
                        return "", fmt.Errorf("failed to get indexes: %w", err)
                    }

                    for _, idx := range indexes {
                        backup.WriteString(idx + ";\n")
                    }

                    return backup.String(), nil
                }

                func BackupDatabase(pg *PostgresRepository) (string, error) {
                    var backup strings.Builder

                    backup.WriteString(fmt.Sprintf("-- PostgreSQL database dump\n-- %s\n\n",
time.Now().Format(time.RFC1123)))
                    backup.WriteString("SET statement_timeout = 0;\n")
                    backup.WriteString("SET lock_timeout = 0;\n")
                    backup.WriteString("SET idle_in_transaction_session_timeout = 0;\n")
                    backup.WriteString("SET client_encoding = 'UTF8';\n")
                    backup.WriteString("SET standard_conforming_strings = on;\n")
                    backup.WriteString("SET check_function_bodies = false;\n")
                    backup.WriteString("SET xmloption = content;\n")
                    backup.WriteString("SET client_min_messages = warning;\n")
                    backup.WriteString("SET row_security = off;\n\n")

                    tables, err := pg.Db.Queryx(`

```

```

        SELECT table_name
        FROM information_schema.tables
        WHERE table_schema = 'public'
        ORDER BY table_name`)
    if err != nil {
        log.Println(err)
        return "", fmt.Errorf("failed to get tables: %w", err)
    }
    defer tables.Close()

    var tableNames []string
    for tables.Next() {
        var tableName string
        if err := tables.Scan(&tableName); err != nil {
            log.Println(err)
            return "", fmt.Errorf("failed to scan table name: %w", err)
        }
        tableNames = append(tableNames, tableName)
    }

    for _, table := range tableNames {
        backup.WriteString(fmt.Sprintf("\n--\n--      Table:      %s\n--\n\n",
table))
        tableBackup, err := BackupTable(pg, table)
        if err != nil {
            log.Println(err)
            return "", fmt.Errorf("failed to backup table %s: %w", table,
err)
        }
        backup.WriteString(tableBackup + "\n")
    }

    var fks []string
    err = pg.Db.Select(&fks, `
        SELECT 'ALTER TABLE ' || conrelid::regclass ||
        ' ADD CONSTRAINT ' || conname ||
        ' ' || pg_get_constraintdef(oid) || ';'
        FROM pg_constraint
        WHERE contype = 'f'`)
    if err != nil {
        log.Println(err)
        return "", fmt.Errorf("failed to get foreign keys: %w", err)
    }

    if len(fks) > 0 {
        backup.WriteString("\n--\n-- Foreign keys\n--\n\n")
        for _, fk := range fks {
            backup.WriteString(fk + "\n")
        }
    }

    return backup.String(), nil
}

func ExportTableToExcel(pg *PostgresRepository, tableName string, writer
fyne.URIWriteCloser) error {
    f := excelize.NewFile()
    defer f.Close()

    index, err := f.NewSheet(tableName)
    if err != nil {
        log.Println(err)
        return err
    }

```



```

    }

    entities, err := GetTableEntities(pg, tableName)
    if err != nil {
        log.Println(err)
        return err
    }

    if len(entities) == 0 {
        log.Printf("no data in table %s\n", tableName)
        return fmt.Errorf("no data in table %s", tableName)
    }

    headers := make([]string, 0, len(entities[0]))
    for k := range entities[0] {
        headers = append(headers, k)
    }

    for i, header := range headers {
        cell, _ := excelize.CoordinatesToCellName(i+1, 1)
        f.SetCellValue(tableName, cell, header)
    }

    for rowIdx, entity := range entities {
        for colIdx, header := range headers {
            cell, _ := excelize.CoordinatesToCellName(colIdx+1,
rowIdx+2)
            f.SetCellValue(tableName, cell, entity[header])
        }
    }

    f.SetActiveSheet(index)
    return f.Write(writer)
}

func ExportDatabaseToExcel(pg *PostgresRepository, writer
fyne.URIWriteCloser) error {
    f := excelize.NewFile()
    defer f.Close()

    f.DeleteSheet("Sheet1")

    tables, err := GetTableList(pg)
    if err != nil {
        log.Println(err)
        return err
    }

    for _, table := range tables {
        index, err := f.NewSheet(table)
        if err != nil {
            log.Println(err)
            return err
        }

        entities, err := GetTableEntities(pg, table)
        if err != nil {
            log.Println(err)
            return err
        }

        if len(entities) == 0 {
            continue
        }
    }
}

```

```

    }

    headers := make([]string, 0, len(entities[0]))
    for k := range entities[0] {
        headers = append(headers, k)
    }

    for i, header := range headers {
        cell, _ := excelize.CoordinatesToCellName(i+1, 1)
        f.SetCellValue(table, cell, header)
    }

    for rowIdx, entity := range entities {
        for colIdx, header := range headers {
            cell, _ := excelize.CoordinatesToCellName(colIdx+1,
rowIdx+2)
                f.SetCellValue(table, cell, entity[header])
            }
        }
        f.SetActiveSheet(index)
    }
    return f.Write(writer)
}

func ExportQueryResultsToExcel(results []map[string]interface{}, writer
fyne.URIWriteCloser) error {
    f := excelize.NewFile()
    defer f.Close()

    sheetName := "Query Results"
    index, err := f.NewSheet(sheetName)
    if err != nil {
        log.Println(err)
        return err
    }

    if len(results) == 0 {
        log.Println("no data to export")
        return fmt.Errorf("no data to export")
    }

    headers := make([]string, 0, len(results[0]))
    for k := range results[0] {
        headers = append(headers, k)
    }

    for i, header := range headers {
        cell, _ := excelize.CoordinatesToCellName(i+1, 1)
        f.SetCellValue(sheetName, cell, header)
    }

    for rowIdx, result := range results {
        for colIdx, header := range headers {
            cell, _ := excelize.CoordinatesToCellName(colIdx+1,
rowIdx+2)
                f.SetCellValue(sheetName, cell, result[header])
            }
        }

        f.SetActiveSheet(index)
    }
    return f.Write(writer)
}

```

Файл gui.go:

```
package gui

import (
    "fmt"
    "log"
    "os"
    "sort"
    "strconv"
    "strings"
    "time"

    "fyne.io/fyne/v2"
    "fyne.io/fyne/v2/app"
    "fyne.io/fyne/v2/container"
    "fyne.io/fyne/v2/dialog"
    "fyne.io/fyne/v2/theme"
    "fyne.io/fyne/v2/widget"
    "lab_6/repository"
)

const (
    maxTextLength = 18
    maxCardFields = 4
)

func truncateText(text string) string {
    runes := []rune(text)
    if len(runes) > maxTextLength {
        return string(runes[:maxTextLength-1]) + "..."
    }
    return text
}

func sortEntityKeys(entity map[string]interface{}) []string {
    keys := make([]string, 0, len(entity))
    for k := range entity {
        keys = append(keys, k)
    }
    sort.Strings(keys)
    priority := map[string]int{"id": -2, "name": -1}
    sort.SliceStable(keys, func(i, j int) bool {
        pi, pj := priority[keys[i]], priority[keys[j]]
        if pi != pj {
            return pi < pj
        }
        return keys[i] < keys[j]
    })
    return keys
}

func getEntityTitle(e map[string]interface{}) string {
    if name, ok := e["name"]; ok {
        return truncateText(fmt.Sprintf("%v", name))
    } else if id, ok := e["id"]; ok {
        return fmt.Sprintf("%v", id)
    }
    return "Entity"
}

func getEntitySortKey(e map[string]interface{}) string {
```

```

    if name, ok := e["name"]; ok {
        return fmt.Sprintf("%v", name)
    } else if id, ok := e["id"]; ok {
        if num, err := strconv.Atoi(fmt.Sprintf("%v", id)); err == nil {
            return fmt.Sprintf("%020d", num)
        }
        return fmt.Sprintf("%v", id)
    }
    return ""
}

func formatValue(value interface{}) string {
    switch v := value.(type) {
    case []byte:
        if strings.HasPrefix(string(v), "$") {
            return string(v)
        }
        return string(v)
    case time.Time:
        return v.Format("2006-01-02 15:04:05")
    default:
        return fmt.Sprintf("%v", v)
    }
}

func CreateEntityForm(pg *repository.PostgresRepository, tableName
string, entity map[string]interface{}, isNew bool, onSuccess func())
fyne.CanvasObject {
    form := container.NewVBox()
    fields := make(map[string]*widget.Entry)
    columns, err := repository.GetTableColumns(pg, tableName)
    if err != nil {
        log.Println(err)
        return form
    }

    for _, col := range columns {
        entry := widget.NewEntry()
        if !isNew {
            if val, ok := entity[col.Name]; ok {
                entry.SetText(formatValue(val))
            }
        }
        fields[col.Name] = entry
        form.Add(container.NewVBox(
            widget.NewLabelWithStyle(col.Name, fyne.TextAlignCenter,
fyne.TextStyle{Bold: true}),
            entry,
        ))
    }

    saveBtn := widget.NewButtonWithIcon("Save", theme.DocumentSaveIcon(),
func() {
        values := make(map[string]interface{})
        for col, entry := range fields {
            values[col] = entry.Text
        }
        var err error
        if isNew {
            err = repository.InsertEntity(pg, tableName, values)
        } else {
            err = repository.UpdateEntity(pg, tableName, entity, values)
        }
    })
}

```

```

        if err != nil {
            dialog.ShowError(err, nil)
            log.Println(err)
        } else if onSuccess != nil {
            log.Println("entity saved successfully")
            onSuccess()
        }
    })

    return container.NewBorder(nil, saveBtn, nil, nil,
        container.NewVScroll(form))
    }

    func ShowEntityDetails(pg *repository.PostgresRepository, tableName
string, entity map[string]interface{}, onUpdate func()) fyne.Window {
        title := getEntityTitle(entity)
        w := fyne.CurrentApp().NewWindow(fmt.Sprintf("%s: %s", tableName, title))
        icon, :=
fyne.LoadResourceFromPath("C:\\BSUIR\\sem_6\\DB\\lab_6\\dino.png")
        w.SetIcon(icon)
        content := container.NewVBox()

        for _, key := range sortEntityKeys(entity) {
            content.Add(widget.NewLabelWithStyle(key, fyne.TextAlignLeading,
fyne.TextStyle{Bold: true}))
            content.Add(widget.NewLabel(formatValue(entity[key])))
            content.Add(widget.NewSeparator())
        }

        actions := container.NewVBox(
            widget.NewButtonWithIcon("Edit", theme.DocumentCreateIcon(),
func() {
                editW := fyne.CurrentApp().NewWindow("Edit Entity")
                editW.SetContent(CreateEntityForm(pg, tableName, entity,
false, func() {
                    onUpdate()
                    editW.Close()
                    if w := fyne.CurrentApp().Driver().AllWindows()[0]; w
!= nil {
                        dialog.ShowInformation("Success", "Entity
updated successfully", w)
                    }
                }))
                icon, :=
fyne.LoadResourceFromPath("C:\\BSUIR\\sem_6\\DB\\lab_6\\dino.png")
                editW.SetIcon(icon)
                editW.Resize(fyne.NewSize(600, 400))
                editW.Show()
            },
            widget.NewButtonWithIcon("Delete", theme.DeleteIcon(), func() {
                dialog.ShowConfirm("Delete", "Are you sure you want to delete
this entity?", func(b bool) {
                    if b {
                        err := repository.DeleteEntity(pg, tableName,
entity)

                        if err == nil {
                            onUpdate()
                            w.Close()
                            if parent :=
fyne.CurrentApp().Driver().AllWindows()[0]; parent != nil {
                                log.Println("entity deleted
successfully")

```

```

                                dialog.ShowInformation("Deleted",
"Entity deleted successfully", parent)
                                }
                                } else {
                                    dialog.ShowError(err, w)
                                    log.Println(err)
                                }
                            }
                        }, w)
                    })),
                )

                w.SetContent(container.NewVBox(
                    widget.NewLabelWithStyle("Entity Details", fyne.TextAlignCenter,
fyne.TextStyle{Bold: true}),
                    widget.NewSeparator(),
                    content,
                    actions,
                ))
                w.Resize(fyne.NewSize(400, w.Canvas().Size().Height))
                return w
            }

            func CreateTableForm(pg *repository.PostgresRepository, w fyne.Window)
fyne.CanvasObject {
                form := container.NewVBox()
                tableNameEntry := widget.NewEntry()
                tableNameEntry.SetPlaceholder("Table name")
                form.Add(container.NewVBox(
                    widget.NewLabelWithStyle("Table Name", fyne.TextAlignLeading,
fyne.TextStyle{Bold: true}),
                    tableNameEntry,
                ))

                fieldRows := container.NewVBox()
                addField := func() {
                    fieldNameEntry := widget.NewEntry()
                    typeSelect := widget.NewSelect([]string{"INTEGER", "REAL", "TEXT",
"BOOLEAN", "DATE", "TIMESTAMP", "MONEY"}, nil)
                    typeSelect.PlaceHolder = "Select type"

                    row := container.NewGridWithColumns(2,
                        container.NewVBox(
                            widget.NewLabel("Column Name"),
                            fieldNameEntry,
                        ),
                        container.NewVBox(
                            widget.NewLabel("Data Type"),
                            typeSelect,
                        ),
                    )
                    fieldRows.Add(row)
                }
                addField()
                form.Add(fieldRows)

                buttons := container.NewVBox(
                    widget.NewButtonWithIcon("Add Column", theme.ContentAddIcon()),
                    widget.NewButtonWithIcon("Create Table", theme.DocumentSaveIcon()),
                )
                func() {
                    tableName := tableNameEntry.Text
                    if tableName == "" {

```

```

        dialog.ShowError(fmt.Errorf("table name cannot be
empty"), w)
        return
    }

    var fieldDefs []string
    for _, obj := range fieldRows.Objects {
        row := obj.(*fyne.Container)
        nameEntry
                                                    :=
row.Objects[0].(*fyne.Container).Objects[1].(*widget.Entry)
        typeSelect
                                                    :=
row.Objects[1].(*fyne.Container).Objects[1].(*widget.Select)

        if nameEntry.Text == "" || typeSelect.Selected == "" {
            dialog.ShowError(fmt.Errorf("all columns must
have name and type"), w)
            log.Println("all columns must have name and
type")
            return
        }
        fieldDefs = append(fieldDefs, fmt.Sprintf("%s %s",
nameEntry.Text, typeSelect.Selected))
    }

    if len(fieldDefs) == 0 {
        dialog.ShowError(fmt.Errorf("at least one column is
required"), w)
        log.Println("at least one column is required")
        return
    }

    query := fmt.Sprintf("CREATE TABLE %s (%s);", tableName,
strings.Join(fieldDefs, ", "))
    _, err := pg.Db.Exec(query)
    if err != nil {
        dialog.ShowError(fmt.Errorf("failed to create table:
%w", err), w)
        log.Println("failed to create table")
        return
    }

    dialog.ShowInformation("Success", fmt.Sprintf("Table %s
created", tableName), w)
    w.Close()
    }),
)

return container.NewBorder(
    nil, // Верх
    buttons, // Низ (две кнопки)
    nil, // Лево
    nil, // Право
    container.NewVScroll(form), // Центр (форма с прокруткой)
)
}

func CreateQueryForm(pg *repository.PostgresRepository, w fyne.Window)
fyne.CanvasObject {
    queryNames, err := repository.GetQueryNames()
    if err != nil {
        err := fmt.Errorf("error getting query names: %w", err)
        dialog.ShowError(err, w)
        log.Println(err)
    }
}

```

```

        return widget.NewLabel("Failed to load queries")
    }

    querySelect := widget.NewSelect(queryNames, nil)
    queryEntry := widget.NewMultiLineEntry()
    queryEntry.SetPlaceholder("Enter SQL query here...")
    queryEntry.Wrapping = fyne.TextWrapWord

    querySelect.OnChanged = func(name string) {
        if sql, err := repository.GetQueryByName(name); err == nil {
            queryEntry.SetText(sql)
        }
    }

    saveBtn := widget.NewButton("Save Query", func() {
        if queryEntry.Text == "" {
            err = fmt.Errorf("no query to save")
            dialog.ShowInformation("Error", err.Error(), w)
            log.Println(err)
            return
        }
        nameEntry := widget.NewEntry()
        nameEntry.SetPlaceholder("Query name")
        dialog.ShowForm("Save Query", "Save", "Cancel",
[]*widget.FormItem{
            widget.NewFormItem("Name", nameEntry),
        }, func(b bool) {
            if b && nameEntry.Text != "" {
                if err := repository.SaveQuery(nameEntry.Text,
queryEntry.Text); err != nil {
                    dialog.ShowError(err, w)
                    log.Println(err)
                } else {
                    dialog.ShowInformation("Success", "Query saved",
w)
                    if names, err := repository.GetQueryNames(); err
== nil {
                        querySelect.Options = names
                        querySelect.Refresh()
                    }
                }
            }
        }, w)
    })

    deleteBtn := widget.NewButton("Delete Query", func() {
        if querySelect.Selected == "" {
            dialog.ShowInformation("Error", "No query selected to
delete", w)
            return
        }

        dialog.ShowConfirm("Delete Query",
            fmt.Sprintf("Are you sure you want to delete query '%s'?",
querySelect.Selected),
            func(b bool) {
                if b {
                    err :=
repository.DeleteQuery(querySelect.Selected)
                    if err != nil {
                        dialog.ShowError(err, w)
                        log.Println(err)
                        return

```



```

    }

    if names, err := repository.GetQueryNames(); err
== nil {
        querySelect.Options = names
        querySelect.Selected = ""
        querySelect.Refresh()
        queryEntry.SetText("")
    }
    dialog.ShowInformation("Success", "Query
deleted", w)
    }
    }, w)
})

runBtn := widget.NewButton("Run Query", func() {
    if queryEntry.Text == "" {
        err := fmt.Errorf("no query to save")
        dialog.ShowInformation("Error", err.Error(), w)
        log.Println(err)
        return
    }
    results, err := repository.ExecuteQuery(pg, queryEntry.Text)
    if err != nil {
        dialog.ShowError(err, w)
        log.Println(err)
        return
    }
    resultWindow := fyne.CurrentApp().NewWindow("Query Results")
    cardGrid := container.NewGridWithColumns(3)
    cardScroll := container.NewVScroll(cardGrid)

    exportBtn := widget.NewButtonWithIcon("Export to Excel",
theme.DownloadIcon(), func() {
        dialog.ShowFileSave(func(writer fyne.URIWriteCloser, err
error) {
            if err != nil {
                dialog.ShowError(err, w)
                log.Println(err)
                return
            }
            if writer == nil {
                return
            }
            defer writer.Close()

            if err := repository.ExportQueryResultsToExcel(results, writer); err != nil {
                dialog.ShowError(err, w)
                log.Println(err)
                return
            }

            dialog.ShowInformation("Success", "Results exported
successfully", w)
        }, resultWindow)
    })

    resultWindow.SetContent(container.NewBorder(
        exportBtn,
        nil,
        nil,
        nil,

```

```

        cardScroll,
    ))

    icon,
fyne.LoadResourceFromPath("C:\\BSUIR\\sem_6\\DB\\lab_6\\dino.png")
    resultWindow.SetIcon(icon)
    resultWindow.Resize(fyne.NewSize(1200, 800))

    if len(results) > 0 {
        for _, entity := range results {
            e := entity
            title := getEntityTitle(e)
            info := container.NewVBox()
            keys := sortEntityKeys(e)
            for _, key := range keys {
                text := fmt.Sprintf("%s: %v", key,
formatValue(e[key]))
                label := widget.NewLabel(text)
                label.Wrapping = fyne.TextTruncate
                info.Add(label)
            }
            card := widget.NewCard(title, "", info)
            cardGrid.Add(card)
        }
    } else {
        cardGrid.Add(widget.NewLabel("No results returned"))
    }

    resultWindow.Show()
})

buttons := container.NewGridWithColumns(3,
    saveBtn,
    deleteBtn,
    runBtn,
)

content := container.NewBorder(
    container.NewVBox(
        widget.NewLabel("Saved Queries:"),
        querySelect,
        widget.NewSeparator(),
        widget.NewLabel("Query:"),
    ),
    buttons,
    nil,
    nil,
    container.NewMax(
        container.NewBorder(
            nil,
            nil,
            nil,
            nil,
            queryEntry,
        ),
    ),
)
return content
}

func LoadTableEntities(pg *repository.PostgresRepository, tableName
string, cardGrid *fyne.Container, w fyne.Window, a fyne.App, onEdit
func(func()), onDelete func(func())) {

```

```

    entities, err := repository.GetTableEntities(pg, tableName)
    if err != nil {
        dialog.ShowError(err, w)
        log.Println(err)
        return
    }
    cardGrid.Objects = nil

    if len(entities) == 0 {
        cardGrid.Add(widget.NewLabel("No entities found"))
        return
    }

    sort.SliceStable(entities, func(i, j int) bool {
        return
            getEntitySortKey(entities[i]) <
getEntitySortKey(entities[j])
    })

    for _, entity := range entities {
        e := entity
        title := getEntityTitle(e)
        info := container.NewVBox()
        keys := sortEntityKeys(e)
        for i, key := range keys {
            if i >= maxCardFields {
                break
            }
            text := fmt.Sprintf("%s: %v", key, formatValue(e[key]))
            label := widget.NewLabel(text)
            label.Wrapping = fyne.TextTruncate
            info.Add(label)
        }
        btnMore := widget.NewButtonWithIcon("More",
theme.MoreHorizontalIcon(), func() {
            ShowEntityDetails(pg, tableName, e, func() {
                LoadTableEntities(pg, tableName, cardGrid, w, a,
onEdit, onDelete)
            }).Show()
        })
        btnEdit := widget.NewButtonWithIcon("Edit",
theme.DocumentCreateIcon(), func() {
            editW := a.NewWindow("Edit Entity")
            editW.SetContent(CreateEntityForm(pg, tableName, e, false,
func() {
                LoadTableEntities(pg, tableName, cardGrid, w, a,
onEdit, onDelete)
            }, editW.Close())
            if parent := fyne.CurrentApp().Driver().AllWindows()[0]; parent != nil {
                dialog.ShowInformation("Success", "Entity
updated successfully", parent)
            }
        })
        icon, := fyne.LoadResourceFromPath("C:\\BSUIR\\sem_6\\DB\\lab_6\\dino.png")
        editW.SetIcon(icon)
        editW.Resize(fyne.NewSize(600, 400))
        editW.Show()
    })
    btnDelete := widget.NewButtonWithIcon("Delete",
theme.DeleteIcon(), func() {
        dialog.ShowConfirm("Delete", "Are you sure you want to delete
this entity?", func(b bool) {

```

```

        if b {
            err := repository.DeleteEntity(pg, tableName, e)
            if err == nil {
                LoadTableEntities(pg, tableName, cardGrid,
w, a, onEdit, onDelete)
                dialog.ShowInformation("Deleted", "Entity
deleted successfully", w)
                log.Println("entity deleted successfully")
            } else {
                dialog.ShowError(err, w)
                log.Println(err)
            }
        }
    }, w)
    })
    actions := container.NewVBox(btnMore, btnEdit, btnDelete)
    card := widget.NewCard(title, "", container.NewVBox(info,
widget.NewSeparator(), actions))
    cardGrid.Add(card)
}
cardGrid.Refresh()
}

func GUI(pg *repository.PostgresRepository) (fyne.App, fyne.Window) {
    os.Setenv("LANGUAGE", "en_US")
    os.Setenv("LANG", "en_US.UTF-8")
    os.Setenv("LC_ALL", "en_US.UTF-8")

    a := app.NewWithID("com.jurassicpark.database")
    w := a.NewWindow("Jurassic Park Database")
    w.Resize(fyne.NewSize(1200, 900))
    a.Preferences().SetString("language", "en")
    icon,
fyne.LoadResourceFromPath("C:\\BSUIR\\sem_6\\DB\\lab_6\\dino.png")
w.SetIcon(icon)

    cardGrid := container.NewGridWithColumns(3)
    cardScroll := container.NewVScroll(cardGrid)

    var selectedTable string
    tables, err := repository.GetTableList(pg)
    if err != nil {
        dialog.ShowError(err, w)
        log.Println(err)
    }

    tableList := widget.NewList(
        func() int { return len(tables) },
        func() fyne.CanvasObject { return widget.NewLabel("") },
        func(i widget.ListItemID, o fyne.CanvasObject) {
            o.(*widget.Label).SetText(tables[i])
        },
    )
    tableList.OnSelected = func(id widget.ListItemID) {
        selectedTable = tables[id]
        LoadTableEntities(pg, selectedTable, cardGrid, w, a, nil, nil)
    }

    backupDatabase := func(isTableBackup bool) {
        dialog.ShowFileSave(func(writer fyne.URIWriteCloser, err error) {
            if err != nil {
                dialog.ShowError(err, w)
                log.Println(err)
            }
        })
    }
}

```

```

        return
    }
    if writer == nil {
        return
    }
    defer writer.Close()

    var backupData string
    var errBackup error
    if isTableBackup && selectedTable != "" {
        backupData, errBackup = repository.BackupTable(pg,
selectedTable)
    } else {
        backupData, errBackup = repository.BackupDatabase(pg)
    }

    if errBackup != nil {
        dialog.ShowError(errBackup, w)
        log.Println(errBackup)
        return
    }

    _, err = writer.Write([]byte(backupData))
    if err != nil {
        dialog.ShowError(err, w)
        log.Println(err)
        return
    }

    dialog.ShowInformation("Success", "Backup completed
successfully", w)
    }, w)
}

exportDatabase := func(isTableExport bool) {
    dialog.ShowFileSave(func(writer fyne.URIWriteCloser, err error) {
        if err != nil {
            dialog.ShowError(err, w)
            log.Println(err)
            return
        }
        if writer == nil {
            return
        }
        defer writer.Close()

        var errExport error
        if isTableExport && selectedTable != "" {
            errExport = repository.ExportTableToExcel(pg,
selectedTable, writer)
        } else {
            errExport = repository.ExportDatabaseToExcel(pg,
writer)
        }

        if errExport != nil {
            dialog.ShowError(errExport, w)
            log.Println(errExport)
            return
        }

        dialog.ShowInformation("Success", "Export completed
successfully", w)

```

```

        }, w)
    }

    newEntityBtn := widget.NewButtonWithIcon("New Entity",
theme.ContentAddIcon(), func() {
    if selectedTable == "" {
        dialog.ShowInformation("Error", "Please select a table
first", w)
        return
    }
    newEntityWindow := a.NewWindow("New Entity")
    icon, :=
fyne.LoadResourceFromPath("C:\\BSUIR\\sem_6\\DB\\lab_6\\dino.png")
    newEntityWindow.SetIcon(icon)
    newEntityWindow.SetContent(CreateEntityForm(pg, selectedTable,
make(map[string]interface{}), true, func() {
        LoadTableEntities(pg, selectedTable, cardGrid, w, a, nil,
nil)
        newEntityWindow.Close()
        dialog.ShowInformation("Success", "Entity created
successfully", w)
    })))
    newEntityWindow.Resize(fyne.NewSize(600, 400))
    newEntityWindow.Show()
})
    newEntityContainer := container.NewBorder(
        nil, nil, nil, nil,
        container.NewMax(newEntityBtn),
    )

    tableButtons := container.NewVBox(
        widget.NewButtonWithIcon("New Table", theme.ContentAddIcon(),
func() {
            newTableWindow := a.NewWindow("Create Table")
            newTableWindow.SetContent(CreateTableForm(pg,
newTableWindow))
            icon, :=
fyne.LoadResourceFromPath("C:\\BSUIR\\sem_6\\DB\\lab_6\\dino.png")
            newTableWindow.SetIcon(icon)
            newTableWindow.Resize(fyne.NewSize(600, 400))
            newTableWindow.SetOnClosed(func() {
                tables, _ = repository.GetTableList(pg)
                tableList.Refresh()
            })
            newTableWindow.Show()
        })),
        widget.NewButtonWithIcon("Delete Table", theme.DeleteIcon(),
func() {
            if selectedTable == "" {
                err := fmt.Errorf("no table selected")
                dialog.ShowInformation("Error", err.Error(), w)
                log.Println(err)
                return
            }
            dialog.ShowConfirm("Delete Table", fmt.Sprintf("Are you sure
you want to delete table %s?", selectedTable), func(b bool) {
                if b {
                    err := repository.DeleteTable(pg, selectedTable)
                    if err == nil {
                        selectedTable = ""
                        tables, _ = repository.GetTableList(pg)
                        tableList.Refresh()
                        cardGrid.Objects = nil

```

```

                                cardGrid.Refresh()
                                dialog.ShowInformation("Deleted", "Table
deleted", w)

                                log.Println("table deleted successfully")
                                } else {
                                    dialog.ShowError(err, w)
                                    log.Println(err)
                                }
                            }
                        }, w)
                    })),
                widget.NewButtonWithIcon("Export Table", theme.DownloadIcon(),
func() {
                    if selectedTable == "" {
                        err := fmt.Errorf("no table selected")
                        dialog.ShowInformation("Error", err.Error(), w)
                        log.Println(err)
                        return
                    }
                    exportDatabase(true)
                })),
                widget.NewButtonWithIcon("Export Database", theme.DownloadIcon(),
func() {
                    exportDatabase(false)
                })),
                widget.NewButtonWithIcon("Backup Table", theme.StorageIcon(),
func() {
                    if selectedTable == "" {
                        err := fmt.Errorf("no table selected")
                        dialog.ShowInformation("Error", err.Error(), w)
                        log.Println(err)
                        return
                    }
                    backupDatabase(true)
                })),
                widget.NewButtonWithIcon("Backup Database", theme.StorageIcon(),
func() {
                    backupDatabase(false)
                })),
                widget.NewButtonWithIcon("Run Query", theme.DocumentIcon(), func()
{
                    queryWindow := a.NewWindow("Run Query")
                    queryWindow.SetContent(CreateQueryForm(pg, queryWindow))
                    icon, :=
fyne.LoadResourceFromPath("C:\\BSUIR\\sem_6\\DB\\lab_6\\dino.png")
                    queryWindow.SetIcon(icon)
                    queryWindow.Resize(fyne.NewSize(600, 400))
                    queryWindow.Show()
                })),
            )

            leftPanel := container.NewBorder(
                widget.NewLabelWithStyle("Tables", fyne.TextAlignCenter,
fyne.TextStyle{Bold: true}),
                container.NewVBox(tableButtons), nil, nil,
                container.NewVScroll(tableList),
            )

            rightContent := container.NewBorder(
                newEntityContainer,
                nil,
                nil,
                nil,

```

```
        cardScroll,  
    )  
  
    split := container.NewHSplit(leftPanel, rightContent)  
    split.Offset = 0.25  
    w.SetContent(split)  
    return a, w  
}
```